TOGGLEMASTER • TECH CHALLENGE FASE 2

Roteiro para dois apresentadores

Revisão documental: 30/10/2026 • alvo 19 minutos • limite 20 minutos.

Dois apresentadores: A conduz as deixas 1–18 (0:00–9:00), seguindo os itens 1, 2 e 3 do enunciado: Docker, infraestrutura AWS e configuração do cluster. B conduz as deixas 19–37 (9:00–19:00), seguindo os itens 4 e 5: manifestos, escalabilidade, demonstrações e encerramento.

Cada apresentador também executa os comandos e troca as telas da sua própria parte. Primeiro prepara a tela e espera o resultado; depois faz a fala. Na deixa 18, A passa a palavra a B, que assume a tela na deixa 19. As duas partes podem ser gravadas separadamente e unidas na edição.

> **Pendência de entrega:** o enunciado exige envio manual de mensagens ao SQS e escala do analytics. A captura atual demonstra escala do evaluation; analytics permaneceu em uma réplica. Ensaiar a deixa 34 e atualizar a fala com o resultado real antes da versão final do vídeo.

Materiais

02_COMANDOS_EM_ORDEM.txt: comandos PowerShell.
04_SLIDES.html/PDF: telas de apresentação.
05_MAPA_DE_TELAS.txt: evidências por deixa.
01_VALIDACAO_E_PENDENCIAS.txt: correções e limites.

Antes de gravar

Preparar credenciais fora da gravação, confirmar o contexto EKS, nós Ready, cinco Deployments disponíveis e métricas dos dois HPAs. Mostrar docker compose up e os nove containers locais. Conferir Jobs e sincronização dos segredos. Testar HTTP e correlacionar evento no DynamoDB.

Usar resultados observados para as falas de sucesso. Exibir as capturas como registros datados. Não mostrar chaves nas respostas, headers ou console do Postman.

Original preservado: ToggleMaster_Fase2_Roteiro_Final_Gravacao.pdf

Sequência do enunciado

Referência: POSTECH - Tech Challenge - Fase 2.pdf, páginas 4–9. A abertura é breve; os requisitos seguem a ordem abaixo.

Item / página / apresentador	Deixas
A • Item 1 — Análise e Conteinerização (p. 4)	01–06
A • Item 2 — Infraestrutura: EKS, ECR, RDS e Redis (pp. 4–5)	07–11
A • Item 2 — DynamoDB, SQS e conexões (p. 6)	12–15
A • Item 3 — Configurando o Cluster (pp. 6–7)	16–18
B • Item 4 — Manifestos básicos (p. 7)	19–23
B • Item 4 — Ingress e boas práticas (pp. 7–8)	24–27
B • Item 5 — Configurando a Escalabilidade (p. 8)	28–30
B • Demonstração de escala e persistência (p. 9)	31–35
B • Arquitetura, desafios e encerramento (p. 9)	36–37

O bloco final retoma a arquitetura e os desafios solicitados na página 9. Os entregáveis do relatório e o limite de 20 minutos estão no checklist 07_ORDEM_DO_ENUNCIADO.txt.

A • Item 1 — Análise e Conteinerização (p. 4)

A — tela e comandos	A — fala
01 • 0:00 • A Slide 1: abertura. Mostrar a sequência dos itens 1 a 5 do enunciado.	Olá! Somos o grupo do ToggleMaster, com os integrantes indicados na tela. Esta apresentação segue a ordem do Tech Challenge: conteinerização, infraestrutura, configuração do cluster, manifestos e escalabilidade. Eu apresento os três primeiros itens; B continua com implantação e demonstrações.
02 • 0:25 • A Item 1, página 4. Identificar as pastas dos cinco microsserviços.	Auth, em Go, cria e valida chaves de API. Flag, em Python, mantém as definições das feature flags. Targeting, também em Python, mantém as regras de segmentação. Evaluation, em Go, calcula a decisão e usa cache Redis. Analytics, em Python, consome eventos do SQS e os persiste no DynamoDB.
03 • 0:50 • A apoio_codigo/auth-service.Dockerfile e flag-service.Dockerfile.	Usamos builds multi-stage. O serviço Go é compilado em um estágio e leva apenas o binário e as dependências de execução para Alpine. Em Python, preparamos as dependências no builder e as copiamos para a imagem final. Os containers das aplicações executam como usuário não root.
04 • 1:15 • A Comando 02: imagens construídas.	Estas são as imagens construídas. A separação dos estágios evita incluir o compilador e arquivos de build na imagem de execução. Os tamanhos devem ser comparados a partir desta saída, sem confundir tamanho comprimido no ECR com tamanho local.
05 • 1:35 • A Comando 01: docker compose up e os nove containers. Slide 4 como apoio.	O ambiente local usa nove containers: cinco aplicações, dois PostgreSQL, Redis e LocalStack. O enunciado cita DynamoDB Local; utilizamos LocalStack para emular tanto DynamoDB quanto SQS. É uma adaptação funcional, que explicitamos na entrega.
06 • 2:05 • A Comando 02: health checks nas portas publicadas.	Os cinco health checks confirmam a disponibilidade dos serviços locais. As portas internas são 8001 a 8005; no computador, o Compose publica por padrão 18001 a 18005.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.

A • Item 2 — Infraestrutura: EKS, ECR, RDS e Redis (pp. 4–5)

A — tela e comandos	A — fala
07 • 2:30 • A Item 2, páginas 4–6. EKS no Console ou evidência 02, com data da coleta.	Na AWS Academy, usamos a conta 533267349027 e a região us-east-1. O cluster se chama ekstogglemaster. As capturas de apoio foram coletadas em 30 de outubro de 2026; o estado ao vivo deve ser conferido antes da gravação.
08 • 3:00 • A Evidência 02: tgmnodegroup, LabRole e escala 1/2/4. Conferir a role no Console.	O Managed Node Group tgmnodegroup usa instâncias t3.medium: mínimo de um nó, dois desejados e máximo de quatro. A capacidade configurada do grupo de nós é diferente da escala automática dos pods feita pelo HPA.
09 • 3:30 • A Evidência 03: ECR e imagens.	Cada serviço tem um repositório ECR privado. O repositório do flag-service foi criado com o nome namespace/lag-service, e esse é o nome utilizado no deployment. O evaluation está na tag v3; os demais, na v1.
10 • 4:00 • A Evidência 04: os três RDS.	Os três PostgreSQL são db-authservice, dbflags e dbtargeting. Na coleta, todos estavam disponíveis, com armazenamento criptografado e acesso público desabilitado. Cada serviço usa o seu banco e esquema, mantendo a separação dos domínios.
11 • 4:40 • A Evidência 05: ElastiCache.	O cluster redisevaluation usa TLS, autenticação por token e cluster mode. O evaluation armazena por 30 segundos os dados combinados de flag e regra. Em um cache hit, evita novas chamadas aos serviços de flags e targeting.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.

A • Item 2 — DynamoDB, SQS e conexões (p. 6)

A — tela e comandos	A — fala
12 • 5:20 • A Evidência 07: definição da tabela DynamoDB. Slide 3 para comparar os armazenamentos.	A tabela ToggleMasterAnalytics guarda os eventos, com event_id como chave de partição. RDS atende os dados relacionais; Redis atende o cache temporário; DynamoDB atende a persistência analítica. No scan registrado, encontramos eventos da flag load-postman-fiap.
13 • 5:50 • A Evidência 06: SQS tgmqueue.	A fila Standard tgmqueue desacopla a avaliação do processamento analítico. O worker remove a mensagem depois de gravar o evento. Idempotência, fila de mensagens mortas e uma estratégia de retry são evoluções importantes para produção.
14 • 6:15 • A Evidência 08: Security Groups e endpoints privados.	Os bancos e o Redis usam endpoints privados. Há regras para o grupo de segurança do EKS nas portas 5432 e 6379. Nos grupos RDS também aparece uma regra administrativa para um IP /32. Portanto, as regras não são exclusivamente do EKS; a existência da regra isoladamente não torna o banco público.
15 • 6:40 • A 06_URLS_E_NOMES_DO_LAB.txt: nomes e endpoints sem senhas.	Concluímos o checklist de infraestrutura anotando endpoints, nome da tabela e identificação da fila. Endereços e configurações não sensíveis vão para ConfigMaps; credenciais são guardadas no Secrets Manager. No Academy usamos a LabRole existente, conforme a opção A do enunciado.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.

A • Item 3 — Configurando o Cluster (pp. 6–7)

A — tela e comandos	A — fala
16 • 7:00 • A Item 3, página 6. Comando 03: Metrics Server, kubectl top nodes e top pods.	Agora configuramos o cluster, começando pelo Metrics Server. Ele fornece as métricas de CPU e memória de nós e pods, necessárias ao HPA. Antes de demonstrar escala, conferimos que as métricas estão disponíveis. A decisão de aumentar réplicas será mostrada no item 5.
17 • 7:45 • A Item 3, páginas 6–7. Comando 03: controller ingress-nginx, Helm e Service; evidência 11.	O segundo componente é o Nginx Ingress Controller, instalado via Helm. O Service publica a entrada externa. Na coleta, o recurso criado era um Classic Load Balancer, enquanto o enunciado menciona ALB ou NLB. Essa divergência está registrada como ponto de adequação. As rotas das aplicações serão mostradas com os manifestos, no item 4.
18 • 8:30 • A A encerra o item 3 e passa a palavra a B às 9:00.	Concluímos Docker, infraestrutura AWS e configuração do cluster, nessa ordem. Agora passo a palavra ao apresentador B, que seguirá com os manifestos, o acesso externo e a escalabilidade exigida no desafio.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.

B • Item 4 — Manifestos básicos (p. 7)

B — tela e comandos	B — fala
19 • 9:00 • B Item 4, páginas 7–8. Comando 04: contexto, namespaces e flag-kustomization.yaml.	Obrigado, A. Começo os manifestos pela ordem do documento: Namespace, Deployment, Service, Secret e ConfigMap. Usamos os namespaces auth-service, flag-service, targeting-service, evaluation-service e analytics-service. O Kustomize aplica esses nomes finais aos recursos de cada serviço.
20 • 9:20 • B apoio_codigo/flag-deployment.yaml: imagem ECR e réplicas.	O Deployment define os pods e a imagem publicada no ECR. Para o flag-service, usamos namespace/lag-service:v1, que é o nome real criado na conta. Os cinco Deployments podem ser atualizados e escalados separadamente.
21 • 9:45 • B Comando 04: Services ClusterIP dos cinco namespaces.	Cada aplicação tem um Service ClusterIP para comunicação dentro do cluster. Um exemplo de DNS é flag-service.flag-service.svc.cluster.local. O Service mantém um endereço estável enquanto os pods são substituídos.
22 • 10:05 • B Comando 04 ou evidências 10 e 12: Secrets/ExternalSecrets, apenas nomes e status.	O Secrets Manager armazena os valores sensíveis e o External Secrets Operator sincroniza os quatro segredos. Os Secrets Kubernetes usam Base64 no campo data: isso é codificação, não criptografia. Mostramos apenas nomes e status. No Academy, o acesso pela LabRole dos nós depende do IMDSv2 com hop limit 2.
23 • 10:35 • B apoio_codigo/evaluation-configmap.yaml: nomes das variáveis e URLs internas.	Os ConfigMaps recebem dados não sensíveis, como região, endpoints internos e URL da fila. Senhas e tokens ficam nos Secrets. Assim, configuração e código ficam separados, e cada Deployment referencia os recursos de que precisa.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.

B • Item 4 — Ingress e boas práticas (pp. 7–8)

B — tela e comandos	B — fala
24 • 11:00 • B Comando 04: Ingress e apoio_codigo/auth-ingress.yaml.	Os manifestos Ingress definem as rotas /auth, /flags, /rules e /evaluate sobre a mesma entrada externa. O controller traduz essas regras em roteamento para os Services. O analytics não tem rota pública porque é um worker interno.
25 • 11:25 • B Item 4: boas práticas. Mostrar probes, resources e securityContext no flag-deployment.yaml.	Os Deployments definem probes, requests e limits. O exemplo de flag executa sem root, sem elevação de privilégios, com capabilities removidas e raiz somente leitura. Readiness decide quando o pod recebe tráfego; liveness permite reiniciar um container que deixou de responder.
26 • 11:50 • B Comando 04: Jobs, Deployments e cinco pods.	Os Jobs versionam a inicialização SQL dos bancos. Conferimos os estados disponíveis no momento da gravação e os cinco microsserviços como pods prontos. Se os Jobs já tiverem sido removidos, mostramos seus manifestos sem afirmar que estão concluídos nesta tela.
27 • 12:10 • B Comando 05 ou Postman: validar chave, criar flag/regra e avaliar pelo Load Balancer.	Para comprovar o Ingress, usamos uma API key preparada sem exibi-la, criamos uma flag de nome exclusivo e uma regra de 100%, e chamamos /evaluate pelo Load Balancer. A resposta esperada é result true. Como o envio ao SQS é assíncrono, a confirmação do fluxo completo exige correlacionar essa flag no DynamoDB.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.

B • Item 5 — Configurando a Escalabilidade (p. 8)

B — tela e comandos	B — fala
28 • 12:50 • B Item 5, página 8. apoio_codigo/evaluation-hpa.yaml e kubectl get hpa.	O HPA do evaluation usa a CPU média em relação ao request. No nosso ambiente, o alvo é 30%, com mínimo de uma e máximo de cinco réplicas. O valor de 70% citado no enunciado é um exemplo; a configuração e a evidência aqui usam 30%.
29 • 13:10 • B apoio_codigo/analytics-hpa.yaml.	O analytics também tem HPA por CPU, na faixa de uma a cinco réplicas. Ele não monitora diretamente o tamanho da fila: para escalar, o processamento precisa elevar a CPU acima do alvo. Ter mensagens pendentes não garante essa condição.
30 • 13:30 • B Item 5: justificar opção Academy; KEDA é alternativa opcional do enunciado.	Seguimos o HPA por CPU previsto para a opção AWS Academy. O enunciado apresenta KEDA como alternativa de escala pela fila em um ambiente com permissões adequadas. Nossa demonstração deve comprovar os dois HPAs; a do analytics ainda precisa ser concluída.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.

B • Demonstração de escala e persistência (p. 9)

B — tela e comandos	B — fala
31 • 13:50 • B Demonstração, página 9. Comando 06; evidência 13.	Na prova do evaluation, dois geradores fizeram chamadas à API. Partimos de uma réplica. Com CPU de 42% para alvo de 30%, capturamos duas de duas réplicas disponíveis e os dois pods Running. Exibimos essa coleta como registro datado do ensaio.
32 • 14:25 • B Evidência 13: encerramento da carga e retorno a 1/1.	A linha do tempo ainda registra três réplicas após o encerramento da carga; depois o evaluation voltou a uma. O resumo maxEvaluation igual a 2 não acompanha esse pico posterior, por isso usamos a sequência completa. O HPA ajusta pods; reduzir instâncias exigiria também autoscaling dos nós.
33 • 15:00 • B Comando 07: enviar mensagens sintéticas manualmente para tgmqueue e anotar RunId.	Em seguida, conforme o enunciado, enviamos mensagens diretamente ao SQS. Cada evento tem user_id, flag_name, result e timestamp. O RunId identifica esse lote sintético, que não deve ser apresentado como tráfego de avaliação HTTP.
34 • 15:25 • B Comando 07: HPA e pods do analytics em watch. PENDENTE: registrar CPU e múltiplas réplicas disponíveis.	Agora verificamos se o processamento elevou a CPU e se o HPA do analytics criou novas réplicas. A evidência existente mostra uma réplica; portanto, este trecho ainda requer ensaio. Só diremos que o analytics escalou após observar múltiplas réplicas disponíveis. Caso contrário, essa exigência permanece pendente.
35 • 16:35 • B Comando 07: filtrar DynamoDB pelo RunId; separar do flag_name do teste HTTP.	Por fim, buscamos no DynamoDB os eventos do mesmo RunId enviado ao SQS. Para a prova HTTP, buscamos o nome exclusivo da flag daquele teste. Essa correlação diferencia os eventos sintéticos das avaliações da API e confirma a persistência pelo analytics.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.

B • Arquitetura, desafios e encerramento (p. 9)

B — tela e comandos	B — fala
36 • 17:20 • B Slides 2, 3 e 6: arquitetura, propósito dos armazenamentos e desafios.	A arquitetura separa cinco responsabilidades. RDS armazena dados relacionais, Redis mantém cache temporário e DynamoDB persiste os eventos analíticos. A fila desacopla avaliação e consumo. Um desafio do Academy foi a recriação dos nós com hop limit 1, que interrompeu o acesso do analytics às credenciais; a restauração autorizada para 2 recuperou o serviço. Esse acesso amplo pela LabRole é uma limitação do laboratório.
37 • 18:40 • B Slide 8: encerramento. Conferir checklist de entregáveis fora da gravação.	Obrigado! Apresentamos a solução seguindo os cinco itens do Tech Challenge. O roteiro, as evidências e os arquivos de reprodução acompanham o projeto. Antes da entrega final, concluiremos a prova de escala do analytics e revisaremos a adequação do balanceador ao enunciado.

O apresentador desta parte também controla a tela. Tempos são metas de edição. Deixa 34 depende de novo ensaio.